Lecture X - Tooling, Git and Your Project

Programming with Python

Dr. Tobias Vlćek
Kühne Logistics University Hamburg - Fall 2026

📋 Checkpoint 5 — Sessions VIII–IX

The first **40 minutes** are the checkpoint. It starts **now**: the acquirer runs one final audit before signing.

- **Individual work**: no neighbors, no chat
- **AI tools are allowed**: being able to **VERIFY** the output is the skill being graded
- The **link and QR** are handed out in class. Open it and start
- 6 short tasks: pull the right number out of a pandas table, fix a line the AI got wrong, and read a chart honestly
- It sweeps **Sessions VIII–IX**: pandas filtering and `groupby`, and telling an honest chart from a misleading one

. . .

When you're done: menu → *Download* → *Download Python code* → upload the `.py` to the **"Checkpoint 5"** assignment on Moodle. **No retakes**: one sitting.

. . .

> ℹ Note
>
> The green ✅ live checks are **provisional**. The final grading runs on our side. The two multiple-choice tasks only say *"recorded"*, not ✅. That's expected, they're scored later. And take a breath: everything in it was rehearsed in the labs.

Episode 10: The Exit

The deal closes

Pens down. The checkpoint is behind you, and so, it turns out, is the startup. Overnight the rival chain **MunchCorp** stopped competing and made an offer: they're **buying the company.**

. . .

The lawyers will tell you deals close on the numbers. This one closed on **yours.** The clean order data you pulled together in the due diligence (Session VIII) and the honest dashboard you built for the pitch (Session IX) are exactly what convinced the acquirer the books were real.

. . .

No cliff-edge growth chart with a chopped axis. Just data that survived a second look. **That's why they signed.**

The handover note

Kevin's job today is to hand over the **data room**: every file the company ever built. He tapes a note to the folder:

. . .

> *"It's all here. And it was never anything fancy — every notebook you downloaded all semester was already a plain `.py` file. Same code, same lines. Open one in a real editor and it just runs."*

. . .

That's the whole reveal, and most of you already suspected it. Those downloads stop being **backups** today and become the **real thing**: on your own machine, in a real editor, under your control.

Your exit package

Every acquisition comes with an exit package. Here's yours: you don't join MunchCorp. You **spin off your own idea** and build it.

. . .

- The tools are the same ones professionals use: **uv**, **Zed**, **git**, **GitHub**
- The work is a real project, built with a partner, over the next weeks
- Today we set up the machine and the repo; the building starts here

. . .

Let's get you kitted out.

The Project

The brief

Over the next weeks you build **one real project** and present it in the final session.

- Work in **pairs**: solo is fine if the numbers don't come out even
- **Deliverable:** a **GitHub repository link**, submitted on Moodle
- The repo's `README.md` carries a short **AI-disclosure section**: which tools you used, what for, and what you **verified yourself**
- You **present in Session XIII**: **10 minutes**, plus **5 minutes** for questions

. . .

> Tip
>
> Pick something you're **actually interested in**. You'll spend real hours on it. Curiosity is the fuel that carries a project past the boring middle.

How it counts

- Project **and** presentation together are worth **40 of the 100** course points
- Your **commit history counts**: it's the evidence of *how* the work happened, not just the final state. Commit small and often (git block, shortly)
- **Sessions XI and XII are supervised work sessions**: we're in the room while you build, so bring your questions

. . .

> Note
>
> The project does not have to be flawless. Show that you scoped it honestly, built it in the open, and can explain your own code. That's the whole bar.

Idea 1 & 2

Data analytics dashboard

- **Core:** load and clean a real CSV with pandas, answer **3 business questions** with charts

Bonus: interactive dashboard (streamlit), automated reporting.

. . .

Web scraping pipeline

- **Core:** scrape **one** site politely into a CSV and analyze it

Bonus: multi-source, change detection, alerts.

Idea 3 & 4

Simulation study

- **Core:** a Monte Carlo simulation of a business decision (pricing, staffing, inventory) with matplotlib results

Bonus: interactive parameters, scenario comparison.

. . .

Automation assistant

- **Core:** a script that does a real repetitive task **end to end**, with error handling and logging

Bonus: scheduling, notifications, a small UI.

Idea 5 & 6

Game

- **Core:** a complete playable **terminal or pygame** game with save/load

Bonus: levels, high scores, procedural content.

. . .

ML-powered app *(ambitious: lean on AI)*

- **Core:** train a **simple** model on tabular data and wrap it in a small interface

Bonus: monitoring, A/B comparison.

Idea 7 & 8

Your startup spin-off

- **Core:** take any system from the course universe (courier routing, review moderation, demand forecasting) and build the **real** version

Bonus: wherever your version wants to grow.

. . .

Your own idea: the open door

- Have something else in mind? **Make it ambitious.** Bring it and we'll discuss scope and feasibility together.

Form your pair, pick your direction

Take ~25 minutes

Right now, in the room:

- **Find your partner** (or decide to go solo if the count is odd). No partner yet? **Come to the front**, we'll match you
- **Pick a direction** from the menu, or bring your own
- Talk it through: what's the *core* you can definitely finish, what's a *bonus* if time allows?

. . .

> Tip
>
> **A good outcome by the end of this block:** a **one-sentence pitch** ("we're building X that does Y for Z"), a **name** for the project, and a decision on **who owns the shared repo** (the owner creates it in the git block; the partner clones). The sentence keeps you honest about scope; the name goes on the repo in a minute.

Your Toolchain

Check your install

Before we build anything, the installs from the Session IX homework need to work. In a terminal:

```
uv --version
git --version
gh --version
```

- Three version numbers? You're set.
- Zed opens when you launch it? Good.

. . .

> ! Important
>
> **Didn't work?** Don't burn the session fighting it. **Flag it for office hours**, pair up with your partner's working machine, and **follow along on the slides** for now. Nobody gets left behind.

Start the project: `uv init`

Pick the folder where you keep course work, then create the project (name it after your pitch):

```
uv init my-project
cd my-project
```

`uv init` creates `main.py`, `pyproject.toml`, `.python-version`, `.gitignore`, and `README.md`.

. . .

> ℹ Note
>
> As long as git is installed (you confirmed that a moment ago with `git --version`), it also **already turns the folder into a git repository**: there's a hidden `.git/` inside. You do **not** run `git init` yourself. Your project is version-controlled from its very first second.

Open a notebook in Zed

Here's the reveal made concrete. Grab **a lab `.py` you downloaded in Part I**: we'll use `nb_03_lab_functions.py`. Copy it **into your project folder** first (in Finder/Explorer, or drop it onto Zed's file tree; dropping on the editor just opens a buffer, it doesn't move the file). Then open it in Zed: it's just code. Read it.

The file needs its tools first. Adding a package to a project is one command:

```
uv add marimo
```

Now run it, no browser required:

```
uv run python nb_03_lab_functions.py
```

...

`uv run` uses the project's own Python and packages. The file that lived in a browser tab all semester now runs on **your machine, from your editor.** That's the whole point of today. *(The Part-II pandas labs also want* `uv add pandas` *and their data file next to them; stick with a Part-I lab today.)*

Connect your AI

Part III **encourages** AI. You've earned the co-pilot. Connect one provider in Zed following the AI-tools guide (the free **Mistral** key is the guaranteed path; paste it into Zed's assistant settings).

...

> ! Important
>
> The disclosure rule still stands, now in writing: your repo's `README.md` says **which tools you used, what for, and what you checked yourself.** AI drafts; **you verify**, same reflex as the checkpoint you just sat.

Git

The five words

Git has a small vocabulary. Learn these five and you can do everything this project needs:

- **Repository**: a folder git watches, with its whole history
- **Commit**: one saved snapshot, with a short message
- **Push**: send your commits up to GitHub
- **Pull**: bring your partner's commits down to you
- **Clone**: make your own copy of a repo from GitHub

...

We'll do each one **live, in your real project repo**. Mistakes are cheap when the repo is two minutes old.

Once per machine

Before any push reaches GitHub, your machine has to prove it's allowed to. Signing in to **Zed does not do this**; git needs its own setup. You need a **(free) GitHub account** and

the `gh` tool. Not installed? Get it via the Git Basics page first. Then, once per computer, in a terminal:

```
gh auth login
```

Choose **GitHub.com**, **HTTPS**, authenticate in the **browser**, and answer **yes** to *"Authenticate Git with your GitHub credentials?"*. Then, once:

```
git config --global pull.rebase false
```

...

> **i** Note
>
> Full walkthrough (installing `gh`, checking it worked) is on the Git Basics page, your at-home reference for all of this.

...

> **!** Important
>
> **Auth fighting you?** Same rule as the toolchain: **flag it**, follow along on the slides, and finish the push at home with the Git Basics page, or with us in the supervised sessions. Nobody's project stalls on a login.

Your first commit

Your `uv init` folder already has files worth saving. In Zed's **command palette** (`Cmd/Ctrl+Shift+P`):

- **`git: stage all`**: mark everything as part of the snapshot
- **`git: commit`** (`Cmd/Ctrl+Enter`): type a message, save it

...

The same thing in the terminal, the words you'll see everywhere:

```
git add .
git commit -m "Initial project setup"
```

A commit lives **only on your computer** so far. Next we send it to GitHub.

Put it on GitHub — repo owner only

This slide is for the repo owner you picked in the pairing block. **Partner: watch**. You'll clone in a minute; **don't create your own GitHub repo.**

GitHub doesn't know about your project yet. Three steps, done by the owner:

1. On **github.com**, create a new **empty** repository (no README, keep it empty). *(There's no "publish" button inside Zed; the GitHub-side repo is made on the website.)*
2. In Zed: **`git: create remote`**, paste the repo's **HTTPS URL** (`https://github.com/you/project.git`); if it asks for a remote name, use `origin` (terminal: `git remote add origin <HTTPS url>`)
3. Do the **first push** from the terminal, this exact line, once:

```
git push -u origin HEAD
```

. . .

After this one-time command, every later push is just **`git: push`** in Zed (terminal: `git push`).

Your partner joins

The owner invites the partner (GitHub → **Settings** → **Collaborators**). The **partner** gets onto the project by cloning it **into a fresh folder**, not inside the practice project from the toolchain block (set that one aside; the clone is your real working copy):

- In Zed: **`git: clone`**, paste the same **HTTPS URL**

```
git clone https://github.com/you/project.git
```

. . .

Cloning brings the whole project and its history, and sets up the connection automatically. The partner pushes and pulls normally from the first moment. Now **both of you pull** (`git: pull` / `git pull`) to confirm you're in sync.

. . .

> **Note**
>
> **Clock ran out?** This step keeps: inviting and cloning works just as well at the start of **Session XI**, with us in the room.

How we work together

Two people, one repo. Four habits keep it painless:

- **Pull before you start.** `git: pull` first, every session: this alone prevents almost every problem
- **Commit small and often.** Little snapshots with honest messages beat one giant one
- **A conflict? Call me.** If git marks a clash, don't fix it alone in week one — bring it to class
- **Never force anything.** No `--force`, no "force push", ever

. . .

> **Note**
>
> Everything today (the actions, the terminal equivalents, the setup, and a **cheatsheet table**) lives on the Git Basics page. When today's follow-along is a fading memory, that's the page you reopen.

Send-off

The road to the finish

- **Sessions XI and XII**: supervised work sessions. We're in the room; you build, we help. Pull before you start, push when you stop.
- **Session XIII**: presentations. **10 minutes** each, **5 minutes** of questions. Show the idea, the build, and one thing that broke and how you fixed it.

. . .

> **Tip**
>
> Between now and then, the repo tells your story. Commit as you go. A steady history is worth more than a heroic final night.

Keep programming

The skill fades if it sits idle. A few honest ways to keep it alive:

- Use Python in your **thesis**: data cleaning, analysis, plots you can defend
- Find a way to **apply it at work**: the repetitive task nobody wants to do by hand
- **Advent of Code**: free, ad-free programming puzzles, one a day from **December 1st**. A genuinely fun way to keep the muscle warm.

Thank you

- We covered the basics of Python: from a first `print` to a real project on your own machine
- You debugged, you verified AI, you shipped honest data
- I hope you enjoyed it, and I hope it's useful long after the grade
- If you have questions or feedback, please tell me

. . .

All the best for your studies and your career. Now go build the thing.

Literature

Books to start with

- Downey, A. B. (2024). Think Python: How to think like a computer scientist (Third edition). O'Reilly. Link to free online version

- Elter, S. (2021). Schrödinger programmiert Python: Das etwas andere Fachbuch (1. Auflage). Rheinwerk Verlag.

. . .

> **i** Note
>
> Setting up the toolchain? The Installing Python, AI Tools and Git Basics pages are the reference versions of everything we did live today.

. . .

For more, see the literature list of this course.